ZIM School - Use Cases

02. Schools: schools, school years, divisions, principals, teachers and the calendar

Modules: Schools
17 use cases (changes) · 9 views and reports
Written from the use case commands, handlers, policies and rules in `src/` on 2026-11-10.

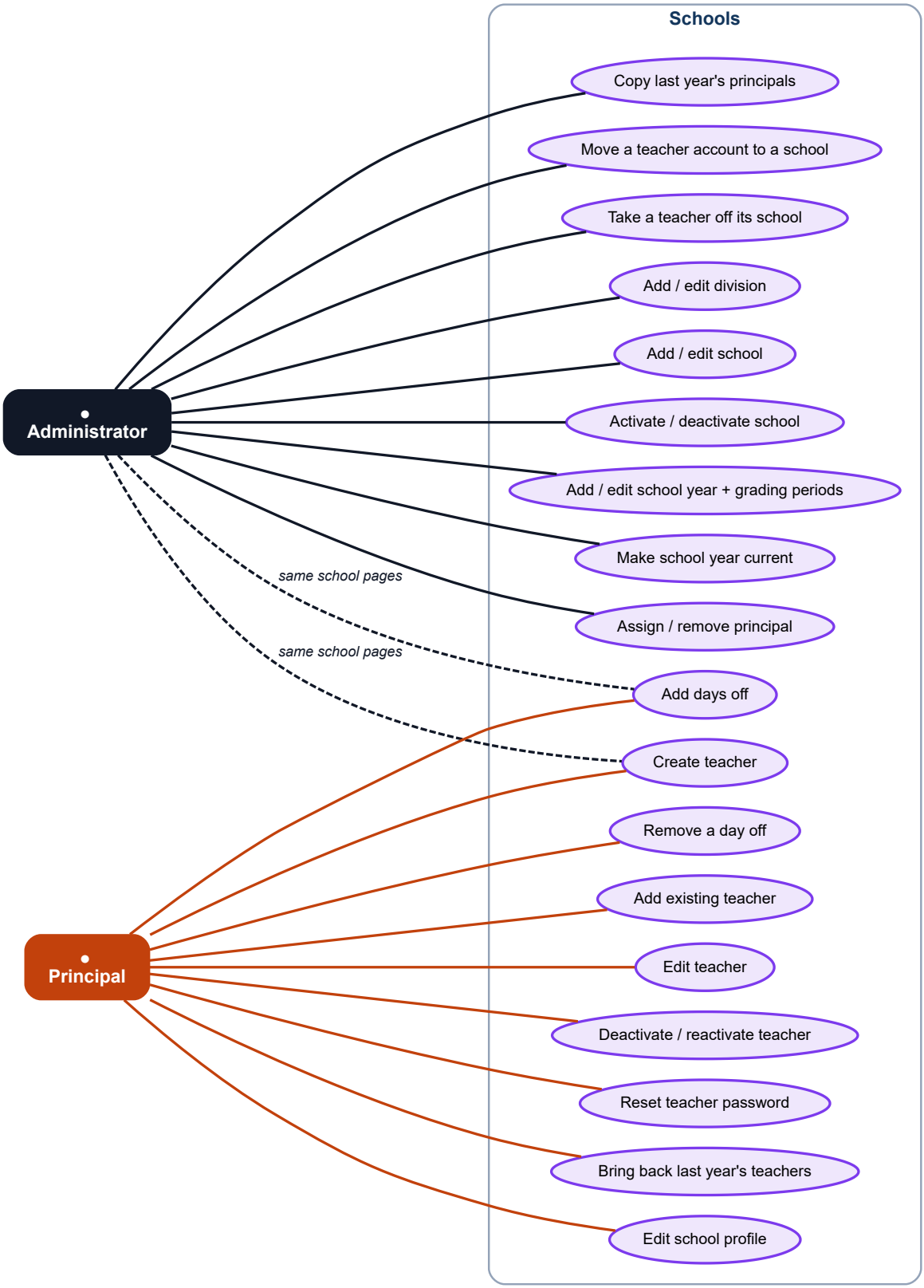
1. What this area does

The Schools module sets up **where** and **when** work happens. The administrator keeps the list of divisions, the schools (with their plan and seats) and the school years with their grading periods, and assigns each school's principal per school year. The principal (or an administrator on the same pages) keeps the school's teachers, its calendar of days off and its own profile.

Plan limits apply everywhere: a school's teachers and students are capped by *plan x seats*; an inactive school or an expired plan makes the school's pages view-only ("The plan of <school> expired on <date>. Contact the administrator to renew."). Who may manage a school is `SchoolVoter::MANAGE`: administrators for any school, principals for their own school in the current year. Teacher changes take the `school-teachers-<id>` lock.

2. Use case map

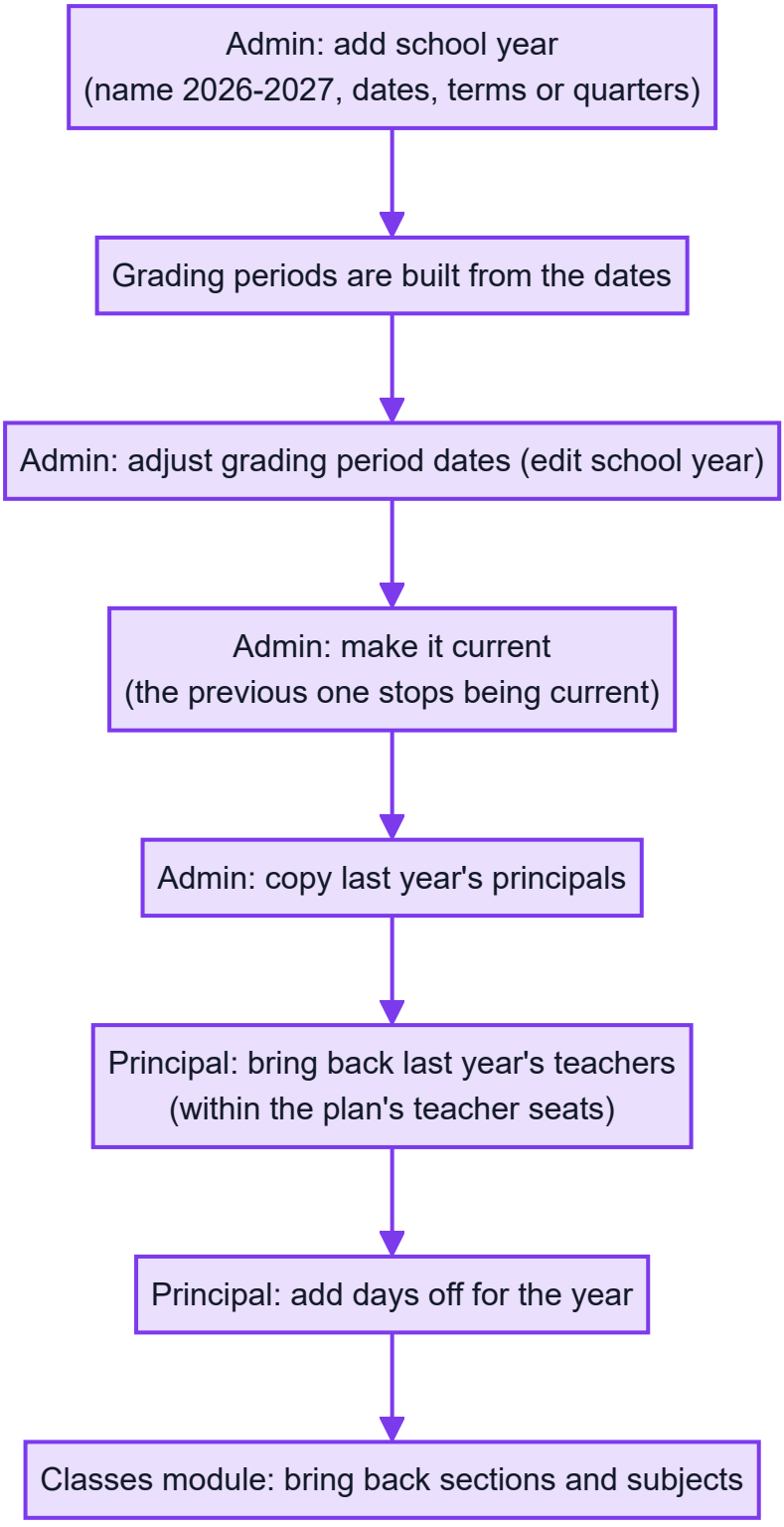
A standard use case diagram: the actors on the left, each use case as an ellipse inside the system frame, and a line for each use case an actor starts. A dashed line has a note on when it applies.



Administrator Principal Line colour = the actor who starts it.

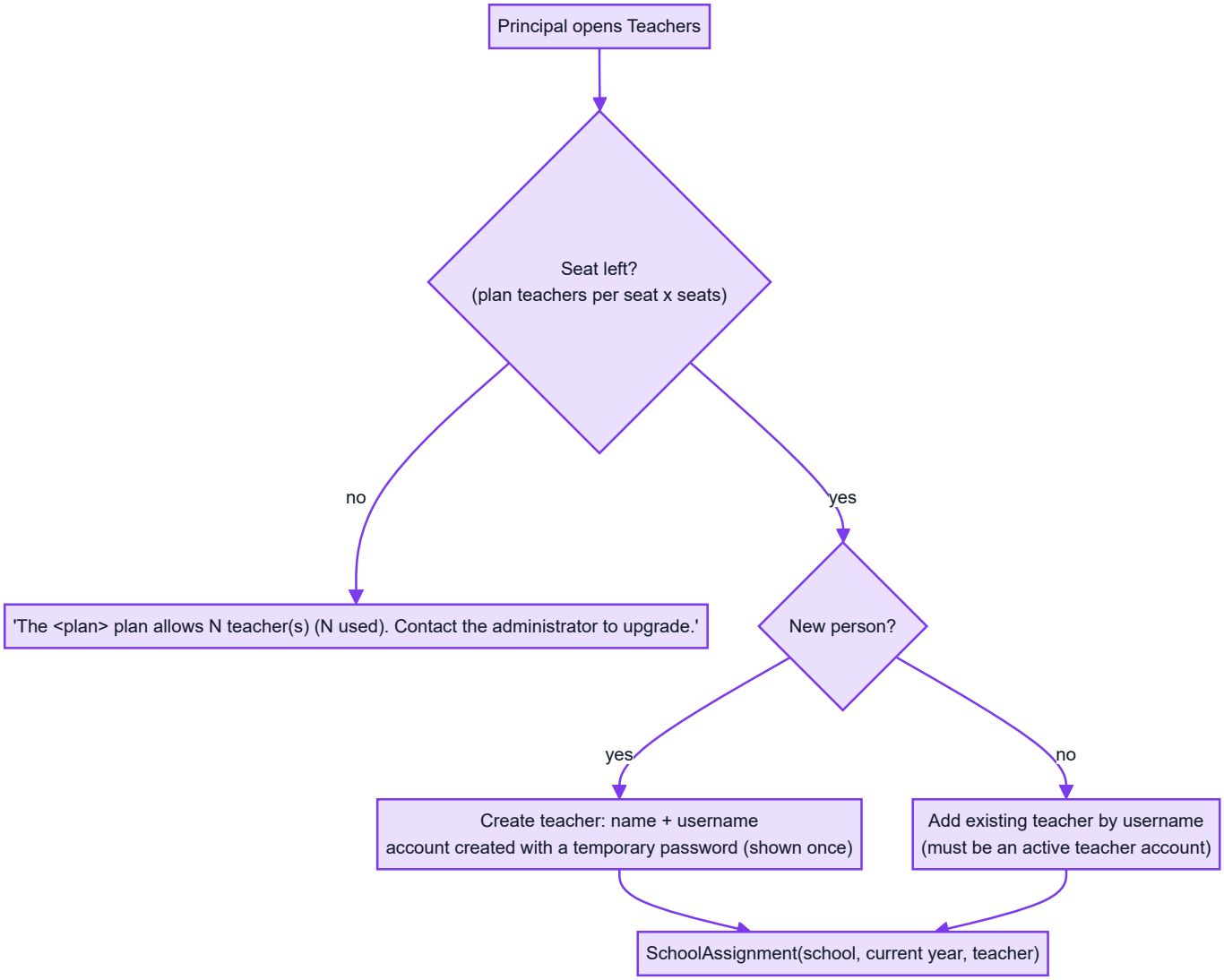
Schools Ellipse colour = the module the use case belongs to.

3.1 Flow: Setting up a new school year



Schools Drawn in the colour of the module this flow belongs to.

3.2 Flow: Adding a teacher within the plan



Schools Drawn in the colour of the module this flow belongs to.

4. Use cases (changes)

Each card is one use case command sent through the use case bus. It runs in one database transaction: if any rule refuses, nothing is saved and the message in *Alternatives and refusals* is shown to the user.

SCH-01 Add or edit a division

Primary actor	Administrator
Goal	Keep the list of schools divisions and their Superintendent (who signs principals' IDs).
Where (route)	`admin_divisions_new /admin/divisions/new; admin_divisions_edit`
Preconditions	• Signed in as administrator.
Main flow	1. The admin picks the region and the division from the fixed Philippine list. 2. Enters the Superintendent's name and position (prefilled "Schools Division Superintendent") and may upload a signature. 3. The system saves the division and logs "Added division ..." / "Saved division ...".
Alternatives and refusals	• Already listed: "This division is already on the list." • Missing fields: "Choose a region.", "Choose a division.", "Enter the Superintendent's ..."
Result (data changed)	Division row; signature image stored as WebP.
In the code	`CreateDivisionCommand, UpdateDivisionCommand (DivisionDetails)`

SCH-02 Add or edit a school

Primary actor	Administrator
Goal	Register a DepEd school and its subscription.
Where (route)	`admin_schools_new /admin/schools/new; admin_schools_edit`
Preconditions	• Signed in as administrator; plans exist.
Main flow	1. The admin enters the 6-digit School ID, name, type, region, division, district and address. 2. Chooses the plan (Basic is preselected), number of seats and the date the plan is valid until. 3. The system saves and logs "Created school ..." / "Updated school ...".
Alternatives and refusals	• School ID not 6 digits: "School ID must be exactly 6 digits."; taken: "This School ID already exists." • Moving to a plan without a principal while the school has one: "Remove the principal of <school> for <year> before choosing the <plan> plan."
Result (data changed)	School row.
In the code	`CreateSchoolCommand, UpdateSchoolCommand (SchoolDetails)`

SCH-03 Activate or deactivate a school

Primary actor	Administrator
Goal	Stop (or resume) a school's use of the system without losing data.
Where (route)	`admin_schools_toggle /admin/schools/{id}/toggle`
Preconditions	• Signed in as administrator.
Main flow	1. The admin presses the toggle on the school list. 2. The system flips School.active and logs it.
Alternatives and refusals	• -
Result (data changed)	School.active toggled. An inactive school's pages become view-only ("This school is inactive.").
In the code	`ToggleSchoolActiveCommand`

SCH-04 Add or edit a school year

Primary actor	Administrator
Goal	Define a school year, its dates and whether it uses terms or quarters.
Where (route)	`admin_school_year_new; admin_school_year_edit (/admin/school-years/...)`
Preconditions	• Signed in as administrator.
Main flow	1. The form suggests the next year's name and dates. 2. The admin sets name (2026-2027), start and end dates and the grading system. 3. The grading periods are built from the dates; when editing, each period's dates can be set. 4. The first school year created becomes current automatically. The change is logged.
Alternatives and refusals	• Bad name: "Use the format 2026-2027."; taken: "This school year already exists." • End before start: "The end date must be after the start date." • Missing period dates: "Enter the dates of <period>." • Switching terms/quarters while grading is open anywhere: "Close grading in every school before switching between terms and quarters." • Switching would delete a period that has grades: "<period> has grades. Switching to <n>s would delete them."
Result (data changed)	SchoolYear and GradingPeriod rows.
In the code	`CreateSchoolYearCommand, UpdateSchoolYearCommand (SchoolYearDetails, GradingPeriodInput)`

SCH-05 Make a school year current

Primary actor	Administrator
Goal	Switch the whole platform to a new school year.
Where (route)	`admin_school_year_make_current /admin/school-years/{id}/make-current`
Preconditions	• Signed in as administrator; the year exists.
Main flow	1. The admin presses Make current. 2. The system clears the old current year, sets this one and logs it.
Alternatives and refusals	• -
Result (data changed)	Exactly one SchoolYear.current = true. Every page and change now works on this year; earlier years become read-only history.
In the code	`MakeSchoolYearCurrentCommand`

SCH-06 Assign or remove a school's principal

Primary actor	Administrator
Goal	Put a principal in charge of a school for a school year.
Where (route)	`admin_schools_principal /admin/schools/{id}/principal;` `admin_schools_principal_remove`
Preconditions	• Signed in as administrator. The school's plan includes a principal.
Main flow	1. The admin opens the school's Principal page (it lists active principals not yet assigned that year). 2. Picks a principal and saves. 3. The system creates the principal's SchoolAssignment for that year (or removes it).
Alternatives and refusals	• Account not a principal: "Only accounts with the Principal role can be assigned." • Plan without principal: "The <plan> plan does not include a principal." • "This account is inactive." / "This school is inactive." / "The school already has a principal for <year>."
Result (data changed)	SchoolAssignment (role principal) created or removed.
In the code	`AssignPrincipalCommand, RemovePrincipalCommand → PrincipalAssigner`

SCH-07 Copy last year's principals

Primary actor	Administrator
Goal	Carry every school's principal into the new school year in one step.
Where (route)	`admin_schools_copy_principals /admin/schools/copy-principals`
Preconditions	• Signed in as administrator; a previous school year exists.
Main flow	1. On the school list for a year, the admin presses Copy principals. 2. The system assigns each school's previous principal for this year where the rules allow.
Alternatives and refusals	• "There is no previous school year to copy from."
Result (data changed)	SchoolAssignment rows (principal) for the year.
In the code	`CopyPrincipalsCommand`

SCH-08 Move a teacher account to a school, or take it off

Primary actor	Administrator
Goal	Set which school a teacher account works at this school year.
Where (route)	`admin_account_school /admin/accounts/{id}/school;` `admin_account_school_remove`
Preconditions	• Signed in as administrator; the account has the teacher role; current school year exists.
Main flow	1. From the account, the admin opens School and picks an active school. 2. The system takes the school's teacher lock, checks the seat limit and assigns the teacher (or removes the assignment). 3. When the teacher leaves a school, the Classes module (TeacherDepartureInterface) clears the section they advised there and logs "Removed <user> as adviser of <section>".
Alternatives and refusals	• "Only teacher accounts are assigned to a school here." • Teaching loads left: "<name> still has N loads this school year." (remove them from the timetable first) • Seat limit reached (plan message).
Result (data changed)	SchoolAssignment (role teacher) created, moved or removed.
In the code	`AssignTeacherSchoolCommand, RemoveTeacherSchoolCommand → SchoolTeacherManager`

SCH-09 Edit the school profile

Primary actor	Principal
Goal	Keep the school's contact details, logo, ID signature and the names that print on forms.
Where (route)	`principal_school /principal/school`
Preconditions	• Signed in as the school's principal this year.
Main flow	1. The principal edits address, contact number, e-mail, Facebook page. 2. Sets the SF8 certifier, the service record 'prepared by' and 'certified by', and the lesson plan 'checked by' names and positions. 3. May upload a new logo and an ID signature (stored as WebP). 4. The system saves and logs "Updated school profile ...".
Alternatives and refusals	• Unreadable or too large image (image service messages).
Result (data changed)	School profile columns.
In the code	`UpdateSchoolProfileCommand`

SCH-10 Add days off

Primary actor	Principal, Administrator
Goal	Mark a holiday, suspension or a whole break as days without classes.
Where (route)	`school_calendar /schools/{id}/calendar`
Preconditions	• Manages the school; current school year exists.
Main flow	1. The principal enters a date (or a from-to range) and a reason. 2. The system adds every weekday in the range that is not already a day off, and logs it. 3. If sections already took attendance on those days, the page tells how many (from the Attendance module).
Alternatives and refusals	• "Pick a date.", "Give a reason." • "The end date must be on or after the start date." • "Days off must be inside the school year (<start> - <end>)." • "Add at most N days at a time."
Result (data changed)	SchoolDayOff rows. Days off are skipped by roll calls, SF2 school days and lesson plan weeks.
In the code	`AddDaysOffCommand`

SCH-11 Remove a day off

Primary actor	Principal, Administrator
Goal	Turn a day off back into a school day.
Where (route)	`school_calendar_remove /schools/{id}/calendar/{dayOff}/remove`
Preconditions	• Manages the school.
Main flow	1. The principal presses Remove next to the day. 2. The system deletes it and logs it.
Alternatives and refusals	• "The day off was not found."
Result (data changed)	SchoolDayOff row removed.
In the code	`RemoveDayOffCommand`

SCH-12 Create a teacher

Primary actor	Principal, Administrator
Goal	Give a new teacher an account and a place at the school this year.
Where (route)	`school_teachers_new /schools/{id}/teachers/new`
Preconditions	• Manages the school; a teacher seat is free; the school is active and its plan valid.
Main flow	1. The principal enters full name and username. 2. The system takes the school's teacher lock, checks the seat limit, creates the account with a temporary password and assigns it to the school for the current year. 3. The temporary password is shown once.
Alternatives and refusals	• Username taken: "This username is already taken." • No seat: "The <plan> plan allows N teacher(s) (N used). Contact the administrator to upgrade." • Double submit: "This was already saved by another request..."
Result (data changed)	User (teacher) + SchoolAssignment.
In the code	`CreateTeacherCommand (TeacherDetails) → SchoolTeacherManager::create`

SCH-13 Add an existing teacher

Primary actor	Principal, Administrator
Goal	Bring a teacher who already has an account (e.g. moved from another school) into this school.
Where (route)	`school_teachers_add_existing /schools/{id}/teachers/add-existing`
Preconditions	• Manages the school; seat free.
Main flow	1. The principal types the teacher's username. 2. The system checks the account and the seat limit and assigns it.
Alternatives and refusals	• "No teacher account with that username." • "This account is inactive. Ask the administrator." • Seat limit.
Result (data changed)	SchoolAssignment.
In the code	`AddExistingTeacherCommand`

SCH-14 Edit a teacher

Primary actor	Principal, Administrator
Goal	Correct a teacher's name or username.
Where (route)	`school_teachers_edit /schools/{id}/teachers/{teacher}/edit`
Preconditions	• Manages the school; the teacher belongs to it this year.
Main flow	1. The principal edits and saves; the change is logged.
Alternatives and refusals	• "This teacher does not belong to the school this school year." • Username taken.
Result (data changed)	User updated.
In the code	`UpdateTeacherCommand`

SCH-15 Deactivate or reactivate a teacher

Primary actor	Principal, Administrator
Goal	Stop a teacher from signing in, or let them back within the plan.
Where (route)	`school_teachers_toggle /schools/{id}/teachers/{teacher}/toggle`
Preconditions	• Manages the school.
Main flow	1. The principal presses the toggle. 2. Reactivation checks the seat limit first.
Alternatives and refusals	• Seat limit reached on reactivation.
Result (data changed)	User.active toggled.
In the code	`ToggleTeacherActiveCommand`

SCH-16 Reset a teacher's password

Primary actor	Principal, Administrator
Goal	Help a teacher who forgot their password.
Where (route)	`school_teachers_reset_password /schools/{id}/teachers/{teacher}/reset-password`
Preconditions	• Manages the school; the teacher belongs to it this year.
Main flow	1. The principal presses Reset password. 2. A temporary password is generated, shown once, and must be changed at next sign-in.
Alternatives and refusals	• -
Result (data changed)	User.password, must_change_password.
In the code	`ResetTeacherPasswordCommand`

SCH-17 Bring back last year's teachers

Primary actor	Principal, Administrator
Goal	Re-assign last year's teachers to the new school year in one step.
Where (route)	`school_teachers_bring_back /schools/{id}/teachers/bring-back`
Preconditions	• Manages the school; a previous school year exists.
Main flow	1. The Teachers page lists last year's teachers not yet back. 2. The principal presses Bring back; teachers are added until the seats run out.
Alternatives and refusals	• "There is no previous school year to bring teachers back from." • Seat limit.
Result (data changed)	SchoolAssignment rows for the current year.
In the code	`BringBackTeachersCommand`

5. Views, downloads and reports (read only)

Each row is one query sent through the query bus. Queries change nothing.

Page / file	Who	Route	What it shows	Query
School list (per school year)	Administrator	`admin_schools` `/admin/schools`	Schools with principal, teacher count, plan and validity; switch year with ?sy=.	`ListSchoolsCommand`
School form	Administrator	`admin_schools_new` / `_edit`	The school and the plans to choose from.	`GetSchoolFormCommand`
Principal assignment	Administrator	`admin_schools_principal`	Current principal and assignable principals.	`GetPrincipalAssignmentCommand`
Divisions	Administrator	`admin_divisions` `/admin/divisions`	Every division by region.	`ListDivisionsCommand`, `GetDivisionCommand`
School years	Administrator	`admin_school_years` `/admin/school-years`	Years newest first, how many schools have each period open.	`ListSchoolYearsCommand`, `GetSchoolYearFormCommand`
Teacher's school	Administrator	`admin_account_school`	The teacher's school this year and schools to move to.	`GetTeacherSchoolCommand`
School profile	Principal	`principal_school` `/principal/school`	The principal's school this year.	`GetSchoolProfileCommand`
Teachers	Principal, Administrator	`school_teachers` `/schools/{id}/teachers`	Teachers this year, seats used, who can be brought back.	`ListSchoolTeachersCommand`, `GetSchoolTeacherCommand`, `GetTeacherSeatCommand`
Calendar	Principal, Administrator	`school_calendar` `/schools/{id}/calendar`	Days off this school year.	`GetSchoolCalendarCommand`